

Search

The basic idea of search is to explore all possible solutions before actually doing something in order to solve a problem or puzzle. A search is an important approach to general-purpose problem-solving. Search techniques are the important and pervasive general AI programming technique. It can be said to be a mental exploration of possibilities.

When we make a decision, we cannot be sure that we have made the best one unless we explore various possibilities (or at least, to some degree).

Examples of Search

- Before deciding on a move, a chess player will consider a number of possible moves, weighing up the probable benefits of each.
- When trying to solve the Rubik's cube, a person may try a few moves "in their head" before deciding on the next move.
- To solve a maze, we may follow some path which leads to a dead-end. We then have to go back to a previous choice-point and try an alternative.
- In deciding to travel in Ghana, say from Accra to Kumasi, we would look at different routes on a map before choosing.

How to search

Search is particularly useful for tracking problems for which a "direct" method or algorithm is not known. Typical examples will be the travelling salesperson problem. Many optimisation problems would also require searching methods.

In the early days of AI, it was thought that searching through possibilities was enough. However, in some interesting domains (e.g., chess), there are too many possibilities.

We need to search “intelligently”, i.e., evaluate all the possibilities without having to investigate every single possibility.

Formulating Search Problems

All search problems can be cast into the following general form:

- **Initial state** such as
 - Starting city for a route [travelling problem]
 - A jumbled Rubik’s cube [Rubik’s Cube problem]
- **Goal state** (or a test for goal state) such as
 - Destination city [Travelling problem]
 - The solved Rubik’s cube [Rubik’s cube problem]
- **The permissible operators** such as
 - Go to city X [Travelling problem]
 - Rotate subcube ... [Rubik’s cube problem]

State and state space

Initial state and Goal state are essential in search. So what is a state?

A **state** is one status of the problem. It captures all relevant information about the problem at that status, such as

- A partial route (from starting city to city X1, to city X2, to city X)
- An unsolved Rubik’s cube (the orientation of all sub-cubes of the cube)

State space is the collection of all states – all possible statuses of the problem. Sometimes it is also known as search space. Typical examples include

- All possible routes
- All possible combinations of sub-cubes

Types of search problems

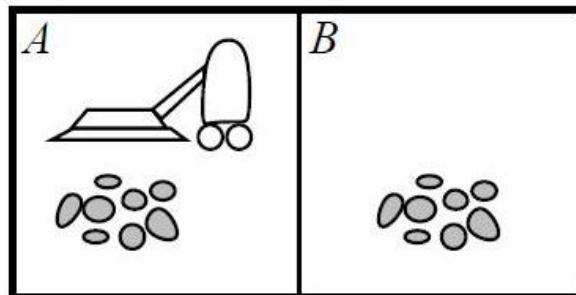
Search problems can be categorised into several groups:

- **Single-state problem:** These are deterministic and fully observable. The next state can be determined exactly (the agent knows exactly what state it will be in) and the solution is a sequence.
- **Conformant problem:** This does not conform to any particular specification. The states are non-observable, implying the agent may have no idea where it is. The solution (if any), is a sequence.
- **Contingency problem:** It is non-deterministic and / or partially observable. The percepts provide new information about current states. The solution is a contingent plan or policy that often searches at intervals before execution. The solution is usually a tree.
- **Exploration problem:** The state space is unknown.

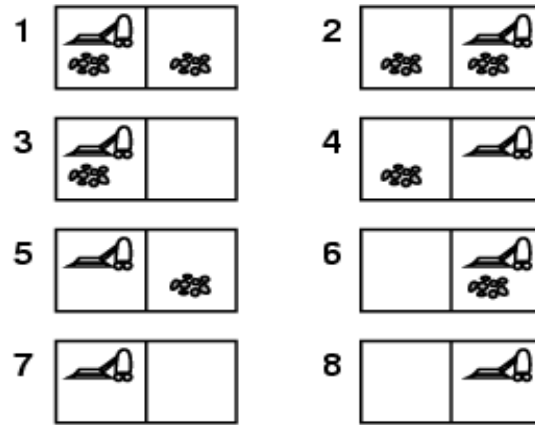
Example: The vacuum world

The vacuum world is a made-up world.

- There are only two squares
- There is an “intelligent” vacuum cleaner which can perceive which square it is in, and whether there is dirt in the square or not.
- The cleaner can choose to move left, move right, suck up the dirt, or do nothing.
- The goal is to have no dirt in both squares.



There are eight possible states in the state space. The goal state is state 7 or 8 (a clean world). The operators are “Move Left”, “Move Right” and “Suck”.



If the initial state is state 5, the solution will go through state 6 and then on to state 8.

If the initial state is state 2, the solution will go through state 4, then to 3, then on to 7.

Example: Cats and Mice Puzzle (C & M)

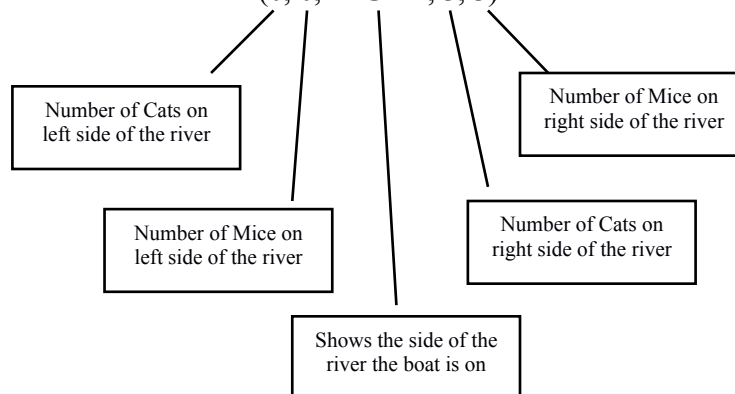
Three cats and three mice come to a crocodile infested river. There is a boat on their side that can be used by one or two “persons”. If cats outnumber the mice at any time, the cats eat mice. How can they use the boat to cross the river so that all mice survive?

The initial state is:

(3, 3, LEFT, 0, 0)

The goal state is:

(0, 0, RIGHT, 3, 3)



Operators for the C & M Problem

A legal move is one which moves up to two “persons” to the opposite bank (such that cats don’t outnumber mice on both sides).

Some possible moves are:

$(3, 3, \text{LEFT}, 0, 0,) \rightarrow (2, 2, \text{RIGHT}, 1, 1)$

$(2, 2, \text{RIGHT}, 1, 1) \rightarrow (2, 3, \text{LEFT}, 1, 0)$

An operator can be written in the format of **Move-1m1c-lr** (Move **1**m mouse and **1** c cat from left bank to right bank)

Move-1m1c-lr brings state $(cleft, mleft, boatpos, cright, mright)$ to

$(cleft-1, mleft-1, \text{RIGHT}, cright+1, mright+1)$.

****Note: Operator **Move-1m1c-lr** has preconditions which are $boatpos=\text{LEFT}$, $cleft \geq 1$, $mleft \geq 1$ and cats don’t outnumber mice on both sides after the move.

State space search of C & M

State space search involves the use of a graph to keep track of the relationships between states. Each node of the graph represents a state of the problem. Each arc in the graph represents the application of an operator in the search process.

The solution is sometimes the sequences of operators which transform the start state to the goal state. It is also sometimes the actual goal state. *A solution is generally defined as a sequence of actions that lead from an initial state to a goal state.*

State space representation

When defining a problem as state space search, the representation of states is critical.

The representation of a state must encapsulate **all the relevant** information necessary to decide “what to do next”, and **none of the irrelevant** information.

- In the C & M problem, the position of the boat is relevant
- Crocodiles or which way the river is flowing are irrelevant.

The initial state, the goal state and the operators should be defined in the representation. Representation varies from problem to problem. A clever representation may reduce computation.

Performing state space search

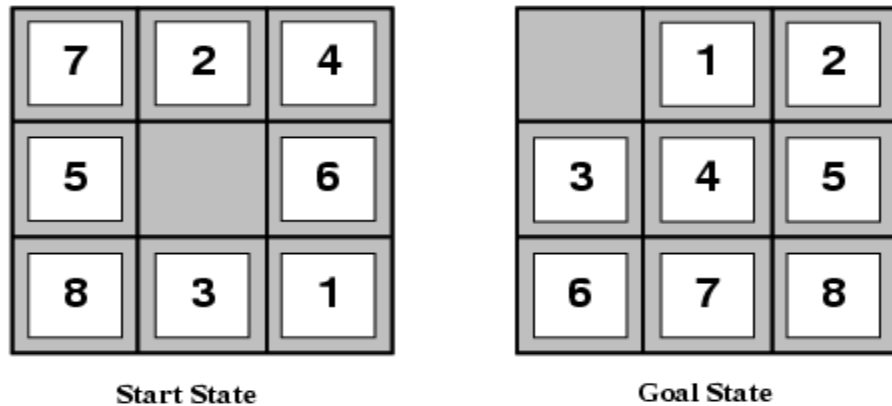
State space search involves finding a path from the initial state of a search problem to a goal state in the state space.

- To do this, we build a *search graph*, starting from the initial state (or the goal state, depending on the strategy).
- We then *expand* a state by applying the operators to that state, generating **ALL** of its successor states.
- The successors are placed in the next level down of the search graph.
- Such a search graph is actually a tree. It is also known as *Search Tree*.
- The order in which we choose states for expansion is determined by the *Search strategy*.
- Different strategies result in (sometimes massively) different behaviour.

Example: The 8-puzzle

This puzzle consists of a 3x3 grid with 8 consecutively numbered tiles arranged on it. Any tile adjacent to the blank grid can be moved to it.

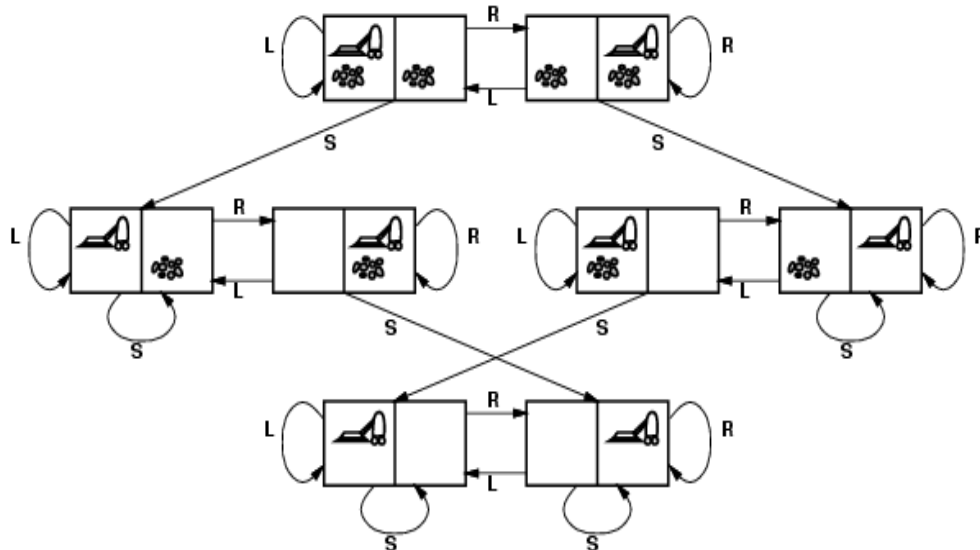
One can choose the initial state and the goal state:



The states can be represented as integer location of tiles. The operators could be represented by a swap blank with left tile, right tile, upper tile or lower tile. This example can have the following attributes:

- **Possible states:** Location of tiles
- **Possible actions:** move blank left, right, up, down
- **Goal test:** goal state (this will be given)
- **Path cost:** GH¢1 per move

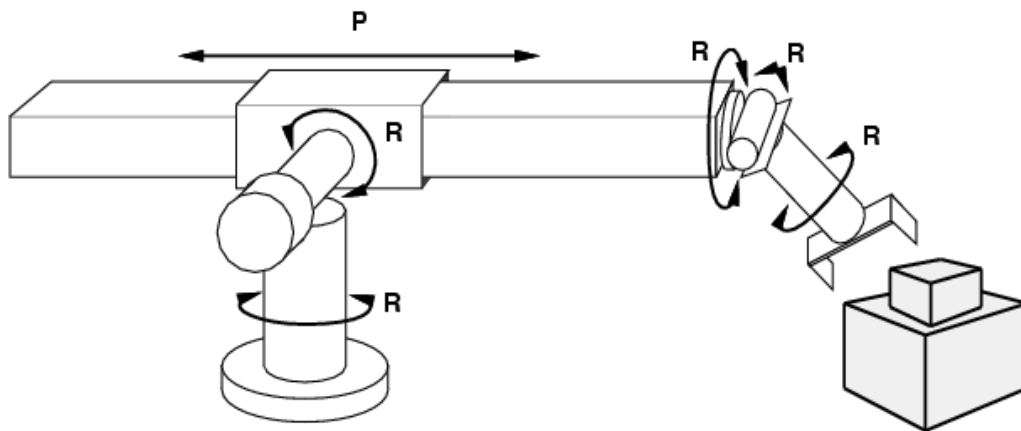
Vacuum World State Space Graph



The graph will have the following attributes:

- **Possible states:** dirt, vacuum locations (we can conveniently ignore dirt amounts, etc)
- **Possible actions:** Left, right, suck, NoOp (No operation)
- **Goal test:** No dirt
- **Path cost:** 1 per action, 0 for NoOp

Example: Robotic Assembly



The robotic assembly can have the following attributes:

- **Possible states:** real-valued coordinates of robot joint angles, parts of the objects to be assembled
- **Possible actions:** continuous motions of robot joints
- **Goal test:** complete assembly
- **Path cost:** time to execute

Search Strategies

Key concept: We want to find the solution while realising in memory as few as possible of the nodes in the state space, which is represented as a search tree. The basic idea is to generate successors of already-explored states. This is the same as node expansion.

A strategy is defined by picking the order of node expansion during a search. The generalised search algorithm is shown as:

```
function Tree-Search( problem, strategy) returns a solution, or failure
  Initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

The implementation of a general tree search can be depicted by the following algorithms:

```
function TREE-SEARCH( problem, fringe) returns a solution, or failure
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE(node)) then return node
    Fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

```
function EXPAND( node, problem) returns a set of nodes
  successors ← the empty set
  for each action, result in SUCCESSOR-FN(problem, STATE[node]) do
    s ← a new NODE
```

```

PARENT-NODE[s] ← node; ACTION[s] ← action; STATE[s] ← result
PATH-COST[s] ← PATH-COST[node] + STEP-COST(node, action, s)
DEPTH[s] ← DEPTH[node] + 1
add s to successors
return successors

```

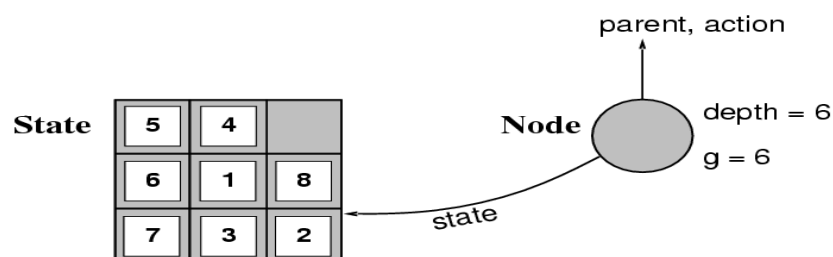
Data structures for search

To facilitate a search algorithm, a data structure is needed for each node to maintain housekeeping information, which can be used during search. The information in a node includes:

- The current state. This is the state in the search space to which the node corresponds
- Parent node of this node
- The operator applied to the parent to generate this node
- The depth of the node (i.e., the number of links from the root to this node)
- The path cost (the cost of the path from the root to this node)

Implementation: States vs. Nodes

- A state is a (representation of) a physical configuration while a node is a data structure constituting part of a search tree.
- Nodes include parent nodes, child nodes, path costs and depth, but states do not have those attributes.
- The EXPAND function creates new nodes and fills the various fields while the SUCCESSOR-FN of the problem creates corresponding states.



Evaluation of search strategies

Search strategies are evaluated along the following four dimensions:

- **Completeness:** Is this strategy guaranteed to find a solution if one exists?
- **Time complexity:** How much time will be used to generate or expand all the expandable nodes?
- **Space complexity:** How much memory (or storage space) is needed to perform the search?
- **Optimality:** In cases where there are several solutions, will the best solution be found first?

*****Often, “the best solution” refers to the solution with the shortest path, implying the least costly solution, if path costs are being applied.

Time and Space Complexity

Time and space complexity are measured in terms of

- ***b***: the maximum branching factor of the search tree
- ***d***: the depth of the search (from root to the solution node)
- ***m***: maximum depth of the state space (***m*** can easily be ∞)

There are two main categories of search strategies: *uninformed search* and *informed search*.

Uninformed Search / Blind Search

These can only distinguish between goal and non-goal states. They do not use information about the number of steps to take from the current state to the goal state. They simply have an initial state, know the goal state they want to achieve, and work towards it without prior knowledge of exactly how many steps will take them there.

The types of uninformed search are:

- Depth-first search

- Breadth-first search
- Uniform cost search
- Depth-limited search
- Iterative deepening search
- Bi-directional search

Informed search / Heuristic search

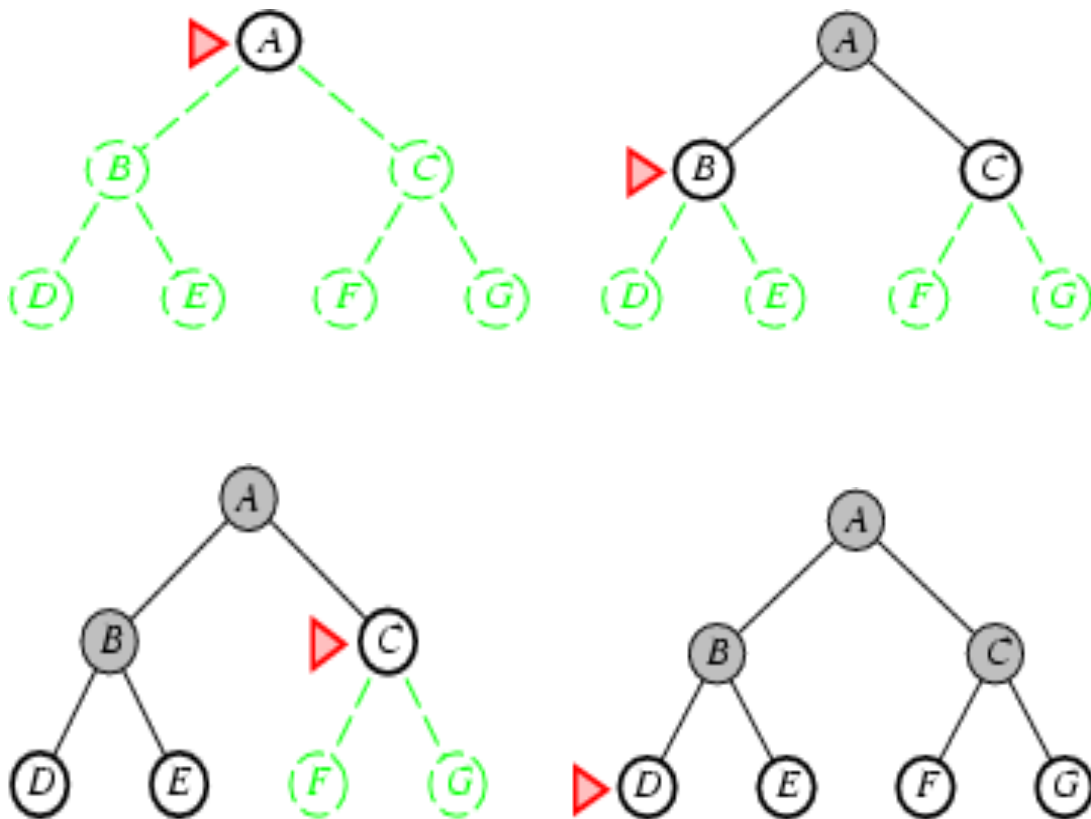
These use guesses or estimates of how far a state is from a goal in deciding which node to expand next. The types in this kind of search include:

- Greedy search (or best-first search)
- A* search

Breadth - First Search

In the breadth first search, the shallowest unexpanded node is expanded. The fringe is a queue (FIFO). This means that new successors are added at the end of the list.

```
/* OPEN and CLOSED are lists of states. */  
OPEN = Start node, CLOSED = empty  
While OPEN is not empty do  
    Remove the first state from OPEN, call it X  
  
    If X is a goal state return success  
  
    Put X on CLOSED  
    Generate all successors of X  
    Eliminate any successors that are already on OPEN or CLOSED  
    Put remaining successors on OPEN  
End while
```



Nodes are explored in the order: A B C D E F G

Properties of Breadth - First Search

- Complete? Yes (if b is finite)
- Time? $1 + b + b^2 + b^3 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$
- Space? $O(b^{d+1})$ (keeps every node in memory)
- Optimal? Yes (if cost = 1 per step)

Suppose $b = 10$, the processing speed is 1000 nodes/sec and each node is 100 bytes, the required time and space for different depths will be as follows:

Depth	Nodes	Time	Memory
0	1	1 millisecond	100 bytes
2	111	.1 seconds	11 kilobytes
4	11,111	11 seconds	1 megabyte
6	10^6	18 minutes	111 megabytes
8	10^8	31 hours	11 gigabytes
10	10^{10}	128 days	1 terabyte
12	10^{12}	35 years	111 terabytes
14	10^{14}	3500 years	11,111 terabytes

Question: Do you think BFS can find the best solution first if there are several solutions?

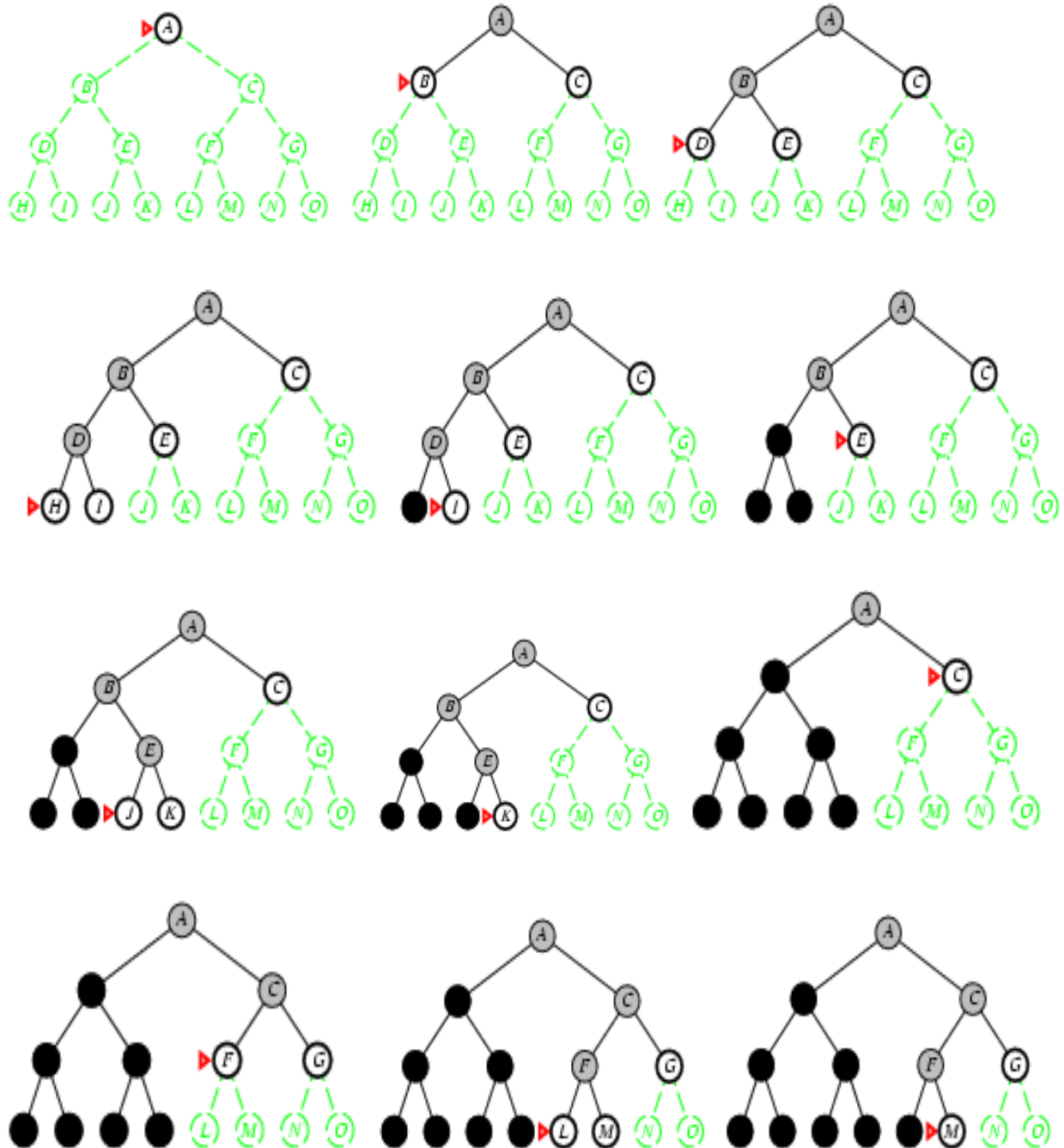
Depth First search

In Depth First search, the deepest unexpanded node is expanded. The list OPEN is a stack (LIFO). This means that successors are added at the front of the list.

```
/* OPEN and CLOSED are lists of states. */
OPEN = Start node, CLOSED = empty
While OPEN is not empty do
    Remove the first state from OPEN, call it X

    If X is a goal state return success

    Put X on CLOSED
    Generate all successors of X
    Eliminate any successors that are already on OPEN or CLOSED
    Put remaining successors on OPEN
End while
```



Nodes are explored in the order: A B D H I E J K C F L M G N O